

Composition and inheritance

Module 3 - Lesson 2

Overview

Inheritance shares behavior across similar classes; composition builds complex objects from simpler ones. Prefer composition, use inheritance for clear is-a relationships.

Key Concepts

- class Child(Parent) inherits methods and attributes from the parent.
- super().__init__() lets subclasses initialize the parent part of themselves.
- Override methods to specialize behavior while keeping the interface.
- Composition means an object holds other objects: a Course has many Lesson objects.
- Composition is more flexible and avoids deep inheritance chains.
- Mixins package small reusable behaviors that can be combined.

Example

```
class BaseUser:
    def __init__(self, name):
        self.name = name
    def permissions(self):
        return []

class Student(BaseUser):
    def permissions(self):
        return ["view_course", "submit_assignment"]

class Course:
    def __init__(self, title):
        self.title = title
        self.lessons = [] # composition
```

Summary

Inheritance specializes behavior for is-a relationships; composition assembles behavior for has-a relationships. Favoring composition keeps hierarchies shallow and flexible.

Practice Checklist

- Create a Student and Instructor that inherit from a BaseUser and override permissions().
- Model a Course that composes multiple Lesson objects.
- Refactor a deep inheritance chain into composition.